

Endsem - Analysis and Design of Algorithms (CSE222)

Name:

Section:

Roll No. :

Important Instruction

Write your solutions precisely within the blank space given in the question paper. You will be given **extra sheet for your rough work**. Anything written outside the given space will be **disregarded** during evaluation. Further, no extra sheet will be provided for writing solutions under any circumstances. Meaningless or irrelevant rambles will fetch zero or even negative marks. However, if you do not know the answer, just write '**I do not know**' to earn 10% of the marks allocated for the question.

Q	1 (10 pts)	2 (10 pts)	3 (20 pts)	4 (10 pts)	5 (10 pts)	Total (60 pts)
Pt						

- Suppose you are given a directed flow network $G = (V, E)$ with an integer capacity function c on the edges. You are also given a maximum flow f^* in this network - that is you are explicitly given the flow values on every edge. Now suppose the capacity of a specific edge $e = (u, v)$ is decreased by exactly 1 unit. Design a *linear* time (i.e., $O(|V| + |E|)$) algorithm to determine the maximum flow in the modified graph starting with the given flow f^* .
 - (6 points) Write a pseudocode **OR** a precise description of the algorithm in English I will give an English description. There can be two cases :
 - e is not saturated by f^* : in this case our algorithm will do nothing and return the same flow f^*
 - e is saturated by f^* : $e = (uv)$. (+2 for this case)
 - Consider the subgraph of $G' = (V, E')$ such that for all edges $e \in E'$, $f_e^* > 0$ (that is edges carrying positive flow). Now run two BFS - one from s until it finds a path to u - call this path P_1 , another from v until it finds a path to t - call this path P_2 . Now decrease the flow along all edge of $P_1 \cup P_2$ and also e by 1. Let the resultant for be f' . (+2 for executing this step.)
 - Run FF algorithm for *one iteration* on $G_{f'}$ and return the resulting flow. (+2 for executing this step correctly.)
 - (4 points) Briefly prove the correctness of your above solution, along with runtime in precise 2-3 formal statements. Meaningless rambles will fetch negative credit.

Correctness. Firstly, the flow cannot increase and can only decrease by at most 1. If e is not saturated by f^* , then decreasing its capacity cannot affect the flow at all since the same flow f^* remains feasible. If e were saturated, then clearly paths P_1 and P_2 exists. The flow value can either decrease by exactly 1 which happens in step (i). However, the flow-value can remain the same as f^* , which happens if *there exists an alternate path* from u to t that *does not use e and may have positive residual capacity*. This is explored by step (ii).
(+1 for mentioning correctness of case 1. +2 for correctness of case 2. Each crucial point carries +0.5)

Runtime. All steps require essentially BFS/DFS a constant number of times. Hence the overall runtime is linear.
(+1 for correct analysis of runtime. Zero credit if algorithm wrong but runtime correct.)

2. Recall the $s - t$ Hamiltonian path (HAM-PATH) problem on a directed graph $G = (V, E)$. The task is to determine a path from vertex s to vertex t that visits every vertex exactly once or report that there is no such path in G (note that this is the ‘search’ version of the problem). We had mentioned in the lecture that HAM-PATH is NP-complete. Now consider the Hamiltonian cycle (HAM-CYCLE) problem. The task is to determine a directed *cycle* that visits *every vertex* in the graph or report that such a cycle does not exist. Show that HAM-CYCLE is NP-complete.

- (a) **(2 points)** Show that HAM-CYCLE is in NP. **Solution.** A candidate solution is an ordering of the vertices. It is straightforward to check in polynomial time if this ordering forms a directed cycle. Hence, HAM-CYCLE is in NP.

+1 for each of the two arguments

- (b) **(8 points)** Show HAM-CYCLE is NP-hard by using an appropriate reduction. Remember to prove both directions. **Solution.** We reduce HAM-PATH to HAM-CYCLE. **Construction.** Suppose we are given an instance $G = (V, E), s, t$ of HAM-PATH. We create an instance G' of HAM-CYCLE as follows - introduce a new vertex, say v and add edges (vs) and (tv) . Now we run our algorithm for HAM-CYCLE - if it returns ‘no cycle’, return ‘no $s-t$ path’. Otherwise, it returns a HAM-CYCLE C . Remove the vertex v along with its entry and exit edge. Return the remaining path.

Proof. ($\implies$ direction) : Suppose there exists an $s - t$ HAM-PATH in G . Then clearly that path along with the two edges vs and (tv) creates a HAM-CYCLE. This is because the HAM-PATH already visits every other vertex in the graph exactly once and the two new edges make sure that v is visited once.

($\impliedby$ direction) : Suppose there exists a HAM-CYCLE in G' . The cycle visits v exactly once by definition. The only way the cycle can reach v is via the edge tv and can exit only via the edge vs . Hence, if we remove these two edge from the cycle, the remaining structure is a path that starts at s , visits all other vertices exactly once and ends at t .

+2 for the construction. +3 for each of the arguments. Deduct 0.5 for missing crucial details - this is subjective and up to the evaluator

Remarks.

- (a) Any other correct reduction from known NP-hard problems would receive full credit. Some of you might have written the solution akin to a mapping reduction that you have studied in TOC - that is YES instances map to YES etc. Surely, they get full credit.
- (b) A common wrong solution could be to not use the dummy vertex and directly put an edge from t to s in the modified graph. One direction still holds - a HAM-PATH in G will surely imply a HAM-CYCLE in G' . But the other direction is not necessarily true. Suppose there is a HAM-CYCLE. But that cycle might not visit s, t consecutively and hence does not imply the existence of an $s - t$ HAM-PATH. You get **score for one direction** in this case.

3. Consider a road network represented by a directed $G = (V, E)$, where vertices represent cities and edges represent the highway connecting two cities. The travel time from city a to city b is t_e , where $e = (a, b) \in E$ is the highway between them. However, one needs to pay a toll tax of b_e , where $b_e > 0$ for traveling along the edge $e \in E$. Design a DP algorithm that runs in $O(|E| \cdot B)$ time to compute the minimum travel time to go from city s to city t considering that you have B amount of money to pay for the tolls. Assume $b_e, \forall e \in E, B$ are integers.

- (a) **(2 points)** Define your subproblems, clearly stating the range for all the parameters used in the subproblems. Loose English descriptions will fetch zero credit.

Solution. For all vertex $v \in V$, and $0 \leq b \leq B$, let

$budSP(v, b) :=$ denote the length of the shortest path from s to v such that the total toll on the edges along the path is at most b .

+2 for fully correct. +1 If missing ranges for the parameters. 0 for complete wrong.

- (b) **(5 points)** Write the recurrences that will help you design your DP solution (remember base cases !)

Solution. Let $\delta^-(v)$ denote the set of vertices $u \in V$ such that there is an edge $uv \in E$. For all $v \in V \setminus \{s\}$, and $1 \leq b \leq B$,

$$budSP(v, b) = \min\left\{\min_{u \in \delta^-(v), b_{uv} \leq b} budSP(u, b - b_{uv}) + t_{uv}, +\infty\right\}$$

Remarks. The $+\infty$ here handles the case where for all $u \in \delta^-(v), b_{uv} > b$ **Base cases.**

$$budSP(s, b) = 0, \quad \forall b, 0 \leq b \leq B,$$

$$budSP(v, 0) = +\infty, \quad \forall v \neq s$$

+3 for recurrence : +3 for general cases, +2 for each base case. Deduct 0.5 for each missing ranges/min etc and missing the $+\infty$ inside the recurrence for general case

- (c) **(1 point)** State the specific subproblem that will give you the final answer to the problem

Solution. $budSP(t, B)$ **+1/0 for this**

- (d) **(5 point)** Write a pseudocode that implements an **iterative solution** based on the recurrences above. If your pseudocode does not match the recurrences in part(b), you will get zero for the pseudocode part.

Algorithm 1 Pseudo-code for Budgeted Shortest $s - t$ Path

```

1: procedure BUDGETEDSP( $G, s, t$ )
2:    $budSP[][]$  : 2D-Array of size  $|V| \times B + 1$ .
3:    $budSP[s][b] := 0, 0 \leq b \leq B$ .
4:    $budSP[v][b] := +\infty, \forall v \neq s, 0 \leq b \leq B$ .
5:   for all  $b$  from 1 to  $B$  do
6:     for all  $v \in V \setminus \{s\}$  do
7:       for all  $u \in \delta^-(v)$  do
8:         if  $b_{uv} \leq b$  then
9:            $budSP[v][b] := \min\{budSP[v][b], budSP[u][b - b_{uv}] + t_{uv}\}$ 
10:        end if
11:      end for
12:    end for
13:  end for
14:  return  $budSP[t][B]$ .
15: end procedure

```

+3 for main loop, +1 for correct initializations. +1 for missing final return statement. -0.5 for each crucial misses

(e) **(2 point)** Argue runtime of your algorithm with reference to the pseudocode you wrote in part (d).

Solution. The outer ‘for’ loop runs for $B + 1$ iterations. Inside the for-loop, there are roughly $\mathcal{O}(|E|)$ operations - same as Bellman-Ford. Hence the runtime is $\mathcal{O}(|E|B)$.

+1 for each of the arguments.

(f) **(5 point)** Will the above algorithm/pseudocode work **as it is** if $b_e = 0$ for some of the edges in the graph? Either say yes or give a counter-example.

Solution. It will not work. In fact, a trivial example is if G has a single edge (st) with $b_{st} = 0, t_{st} = 1$. Then the initialization of $budSP[t][0] = +\infty$ itself is wrong.

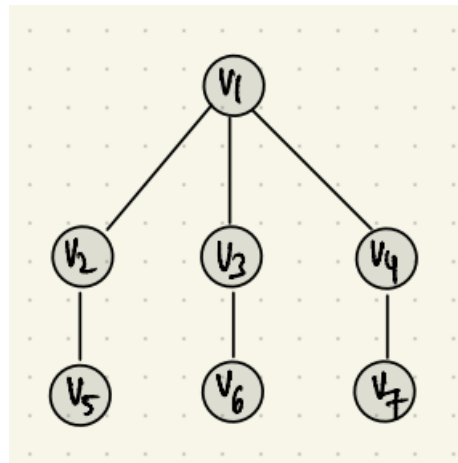
Some of you might have made a more non-trivial example since your pseudocode might have handled the above trivial corner case by starting the main loop from $b = 0$. The following is a counter example for that case. Suppose the graph G is a simple cycle with $t_e = 1$ for all $e \in E$ and $b_e = 0, \forall e \in E$. Further, suppose the distance between s and t is 3 - essentially there are two vertices say u and v between s and t in the cycle - that is there is a sub-path $s \rightarrow u \rightarrow v \rightarrow t$. Now suppose in the inner loop of Line 6, the vertices are being processed in the following order - t, v, u, s . Note that after execution of the outer for-loop for $b = 0$, $budSP[t][0]$ will still remain at $+\infty$ which is wrong. It should be 2.

+2 for a correct example and +3 for explanation.

4. For each part of this question, you will be presented with a problem statement and a greedy algorithm. You need to show via a counter-example that the greedy approach fails for the respective problems

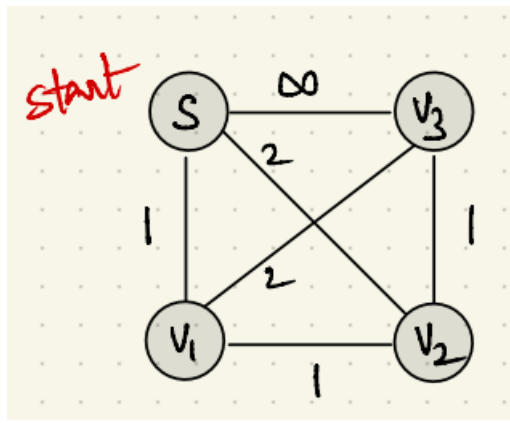
(a) **(5 points) Vertex Cover.** Given an undirected unweighted graph $G = (V, E)$, the vertex cover problem requires one to select a subset of the vertices $V' \subseteq V$ of *minimum possible size* such that every edge in E has at least one of its endpoints in the selected set V' .

Greedy Approach: Start with an empty set V' . Pick the vertex with largest degree and add it to set V' , remove all edges incident on it and repeat the process on the remaining graph. Stop when all edges have been removed. Return V' .



Greedy will choose V_1, V_5, V_6, V_7 while optimal is V_2, V_3, V_4

(b) **(5 points) Traveling Salesman.** Given is an undirected *complete* graph $G = (V, E)$ (that is, every possible edge in the graph is present) with positive length function ℓ on the edges and a start vertex s . The task is to determine a tour (cycle) of *minimum possible length* that visits every vertex exactly once. *Greedy approach:* Start at an arbitrary vertex and mark it ‘current’ and ‘visited’. Repeat the following until all vertices are visited. Select the ‘closest unvisited neighbor’ of current vertex - that is - if the current vertex is v , then choose the *unvisited* neighbor u such that $\ell(v, u)$ is minimum among all neighbors of v ; mark u visited and make it current. Finally, when no vertex is *unvisited*, take the edge connecting the last visited vertex and the start vertex to complete the tour.

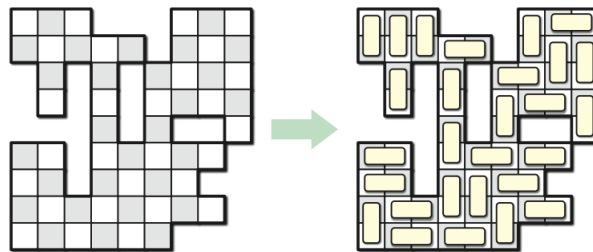


Greedy gives the following ordering : S, V_1, V_2, V_3, S resulting in cost ∞ . Optimal solution is the ordering S, V_1, V_3, V_2, S .

+3 for the correct example (without any explanation). +1 for showing greedy output, +1 for showing optimal

5. Suppose you are given an $n \times n$ checkerboard with some of the squares deleted. You have a large set of dominos which are cards just the right size to cover two adjacent squares of the checkerboard placed either horizontally or vertically. Your input is a boolean matrix $\text{DELETED}[1 \cdots n, 1 \cdots n]$, where $\text{DELETED}[i, j] = \text{True}$ if and only if the square in row i and column j has been deleted. Your output is a YES or NO; you do not have to compute the actual placement of dominos. For example, for the board shown in Figure, your algorithm should return YES.

Design (**5 points**) and prove correctness of (**5 points**) an algorithm (running time not worse than $O(n^4)$) to determine whether one can tile the board with dominos—each domino must cover exactly two undeleted squares, and each undeleted square must be covered by exactly one domino.



Solution. Algorithm. We reduce this to maximum bipartite matching. The construction is as follows. We design a bipartite graph $G = (S_b, S_w, E)$. **+1 for saying bipartite graph**

- (a) each vertex $s \in D_b$ represents a black square and each vertex $s' \in D_w$ represents a white square.
+1 for correctly defining vertices
 (b) an edge between d_i, d_j exists if and only if d_j is neighboring tile of d_i (in other words - left, right, top, bottom with corner cases handled accordingly).
+1 for correctly defining edges

We find the maximum bipartite matching in the graph G . If the matching is a perfect matching (that is, it matches all black squares and white squares), then we return that a domino arrangement is possible, o.w. we say not possible.

+2 for stating the problem (matching/flow) correctly

Correctness. As usual, we show that a valid domino arrangements exists if and only if a perfect matching exists in the graph.

($\implies$ direction) : Suppose a valid domino arrangement exists. We create a perfect matching M in G . Clearly, one domino covers squares d_i, d_j such that - (a) they are of opposite color and (b) they are neighbors. Hence there exists an edge (d_i, d_j) in G - add that edge to M . It is straightforward to observe that the edges form a matching since the only way they do not form a matching is if two dominoes overlap at a square which is not valid in a feasible arrangement. It forms a perfect matching since all squares are covered in a valid arrangement.

+2.5 for this direction. +1 for each of points (a) and (b) written in any legible way (no credit for vague rambles) and 0.5 to finish the argument

(($\impliedby$ direction) : Suppose there exists a perfect matching in G . Then just place the dominos according to the edge in the matching - that is cover d_i, d_j with a single domino if $d_i d_j$ is in the matching. It is again easy to observe that no two dominoes overlap at a square - since they are placed according to the matching. Further, all squares are covered since we start with a perfect matching.

+2.5 for this direction. +1 for defining the valid arrangement, +1 for saying why the arrangement has no overlaps and +0.5 for saying why all squares are covered